

Experiment 7: Depth First Search (DFS)

Aim

To implement the Depth First Search (DFS) algorithm using Python.

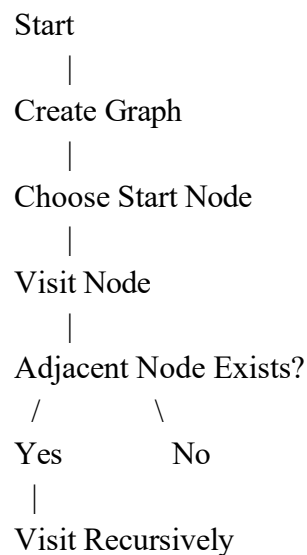
Objective

- To traverse all nodes using recursion.
- To explore graph nodes deeply before backtracking.

Algorithm

1. Create a graph.
2. Select the starting node.
3. Mark it as visited.
4. Visit each adjacent unvisited node recursively.
5. Continue until all nodes are visited.

Flowchart

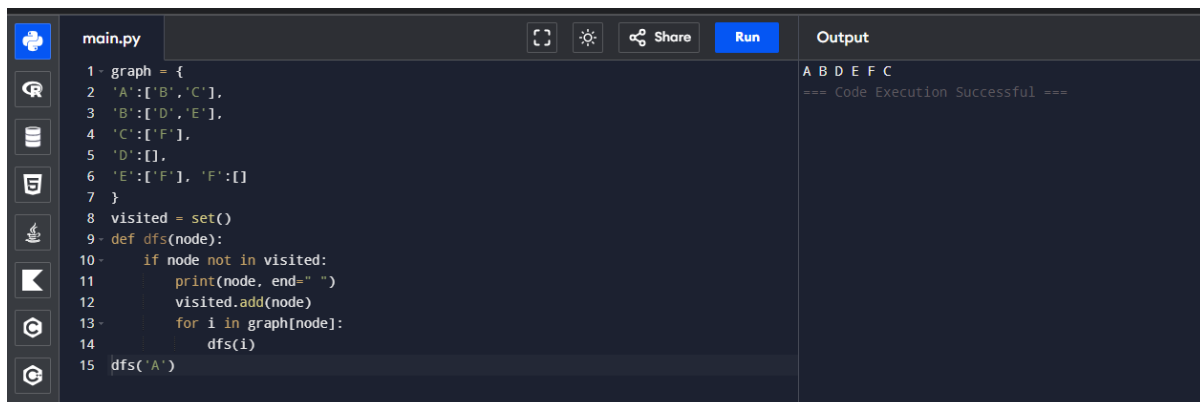


|
Repeat
|
End

Program

```
graph = {  
    'A':['B','C'],  
    'B':['D','E'],  
    'C':['F'],  
    'D':[],  
    'E':['F'], 'F':[]  
}  
  
visited = set() def  
  
dfs(node):  
    if node not in visited: print(node, end=" ")  
        visited.add(node)  
        for i in graph[node]: dfs(i)  
  
dfs('A')
```

Output



The screenshot shows a code editor with a file named 'main.py'. The code defines a graph and a DFS function. The graph has nodes A, B, C, D, E, and F. Node A is connected to B and C. Node B is connected to D and E. Node C is connected to F. Nodes D, E, and F have no outgoing edges. The DFS function starts at node A, prints it, and then recursively visits its neighbors B and C. The output shows the nodes visited in the order A, B, D, E, F, C. The code execution was successful.

```
main.py  
1- graph = {  
2  'A':['B','C'],  
3  'B':['D','E'],  
4  'C':['F'],  
5  'D':[],  
6  'E':['F'], 'F':[]  
7  }  
8  visited = set()  
9  def dfs(node):  
10     if node not in visited:  
11         print(node, end=" ")  
12         visited.add(node)  
13         for i in graph[node]:  
14             dfs(i)  
15  dfs('A')
```

Output
A B D E F C
--- Code Execution Successful ---

Result

The DFS traversal was executed successfully.

Conclusion

DFS explores graph nodes deeply using recursion or a stack.

Result

The BFS traversal was performed successfully.

Conclusion

The Breadth First Search algorithm visits nodes level by level using a queue.